



Informe Técnico

Vulnerabilidades en una red de servicios



Este documento es confidencial y contiene información sensible.
No debería ser impreso o compartido con terceras entidades

13 de Febrero de 2025



Índice

1. Antecedentes	2
2. Objetivo	2
2.1. Alcance	3
2.2. Impedimentos y limitaciones	3
2.3. Resumen general	3
3. Reconocimiento	4
3.1. Enumeración de servicios expuestos	4
3.2. Enumeración de servidores web	5
4. Identificación y explotación de vulnerabilidades	6
4.1. Explotación de servicio SSH	6
4.2. Explotación de posibles servicios	6
5. Contramedidas y buenas practicas	7
5.1. Vulnerabilidad XSS	7
5.2. Vulnerabilidad SQLI	8
5.3. Vulnerabilidad CI	10
5.4. Vulnerabilidad FI	11
6. Conclusión	11



1. Antecedentes

El presente documento recoge los resultados obtenidos durante la fase de auditoría realizada a la maquina **de red de servicios**, enumerando todos los vectores de ataques encontrados, así como los métodos para prevenirlos para cada uno de estos.

Esta maquina ha sido usada desde la tarea anterior entregada en la **Moodle**, una plataforma de entrega de tareas y prácticas para personas en formación.

A continuación, se proporciona el enlace directo a la moodle:

Dirección URL

<https://educacionadistancia.juntadeandalucia.es/centros/cadiz/course/view.php?id=8818>

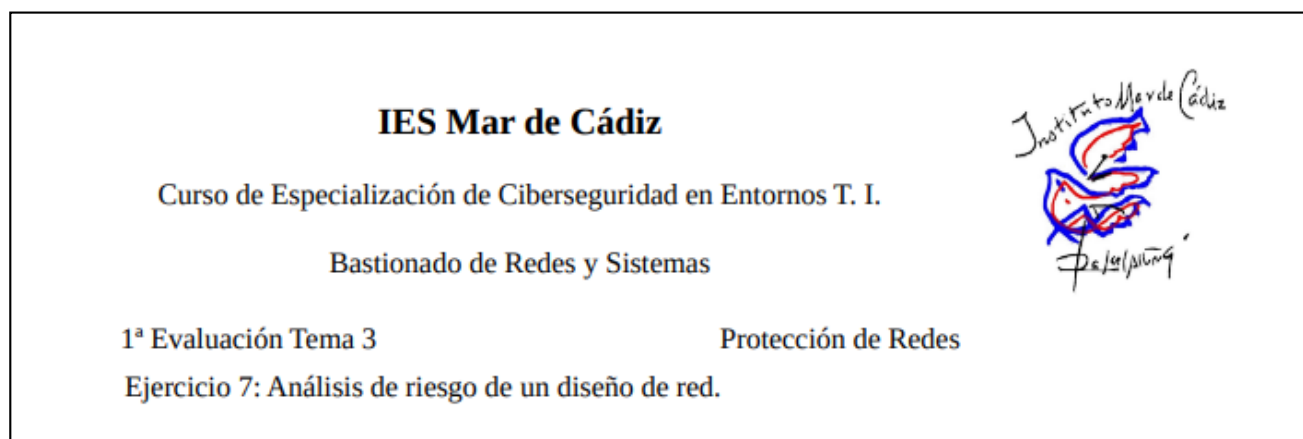


Imagen 1: Encabezado principal de la tarea

2. Objetivo

Los objetivos de la presente auditoria se enfocan en la identificación de posibles vulnerabilidades y debilidades en la maquina **Vulnerabilidades en una red de servicios**, con el propósito de garantizar la integridad y confidencialidad de la información almacenada en ella.

Con este fin se ha llevado acabo un análisis exhaustivo de todos los servicios detectados que se encontraban en dicho servidor, recopilando información detallada de aquellos que representan un riesgo potencial desde el punto de vista de la seguridad informática.



2.1. Alcance

A continuación, se representan los objetivos a cumplir para esta auditoría:

- Identificar los puertos y servicios vulnerables
- Realizar una explotación de las vulnerabilidades encontradas

2.2. Impedimentos y limitaciones

Durante el proceso de auditoría, está terminantemente prohibido realizar algunas de las siguientes actividades:

- Realizar tareas que puedan ocasionar una **denegación de servicios** o afectar a la disponibilidad de los servicios expuestos
- Borrar archivos residentes en el servidor una vez este haya sido vulnerado

2.3. Resumen general

A modo de resumen general, hemos realizado un análisis de los servicios y puertos de la maquina expuesta, mostrando los posibles vectores de ataque como pueden ser las inyecciones XSS o inyecciones SQL.



3. Reconocimiento

3.1. Enumeración de servicios expuestos

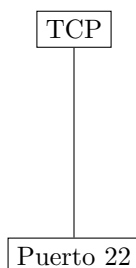
A continuación, se adjunta una evidencia de los puertos y servicios identificados durante el reconocimiento aplicado con la herramienta **nmap**:

```
Starting Nmap 7.93 ( https://nmap.org ) at 2025-03-11 12:47 CET
Nmap scan report for 192.168.179.134
Host is up (0.0000060s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh

Nmap done: 1 IP address (1 host up) scanned in 0.94 seconds
```

Imagen 2: Enumeración de puertos con nmap

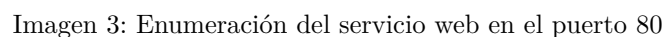
En este caso, se identificaron 1 puerto activo corriendo por el protocolo TCP:



Asimismo, no se encontraron puertos expuestos a través de otros protocolos, por lo que se priorizará la evaluación de los puertos identificados en el primer escaneo efectuado.



Aunque en el escaneo de nmap no se enumerado ningún tipo de puerto de servidor web, como podrían ser el puerto 80 o el puerto 443, si entramos a dicha IP por el puerto 80, veremos un servidor web apache por defecto, dándonos a entender que a priori, el puerto 80 esta completamente asegurado dentro de la red de servicios.



En los resultados obtenidos, es posible identificar las versiones para algunas de las tecnologías existentes

Al ser esta una de las versiones más nuevas, no hay vulnerabilidades conocidas actualmente, por lo que al no estar siendo utilizada por nada, solamente la pagina por defecto, no tiene vulnerabilidades dicho servicio.



4. Identificación y explotación de vulnerabilidades

4.1. Explotación de servicio SSH

Se ha probado con la utilización del usuario anonymous para acceder a dicho servicio, siendo este **NO** un método de acceso. Después de dicho intento fallido se prueba a realizar un ataque de fuerza bruta utilizando **John the Ripper**, es una herramienta de software para descifrar contraseñas encriptadas mediante ataques de diccionario y fuerza bruta.

Una vez pasado un tiempo razonable, John no ha dado ningún resultado correcto, por lo que dicho vector esta seguro.

Con dichos intentos fallidos de acceder a **SSH**, se realiza una captura de trafico de red con wireshark, es una herramienta de análisis de red que permite capturar y examinar el tráfico de datos en tiempo real para diagnosticar problemas y analizar protocolos de comunicación.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.179.1	192.168.179.134	TCP	60	49898 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000540443	VMware_69:92:ba	Broadcast	ARP	42	Who has 192.168.179.1? Tell 192.168.179.134
3	0.000724154	VMware_c0:00:08	VMware_69:92:ba	ARP	60	192.168.179.1 is at 00:50:56:c0:00:08
4	0.000728694	192.168.179.134	192.168.179.1	TCP	60	80 → 49898 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
5	0.001020076	192.168.179.1	192.168.179.134	TCP	60	49898 → 80 [ACK] Seq=1 Ack=1 Win=131328 Len=0
6	0.001264867	192.168.179.1	192.168.179.134	HTTP	569	GET / HTTP/1.1
7	0.001670629	192.168.179.134	192.168.179.1	TCP	54	80 → 49898 [ACK] Seq=1 Ack=516 Win=64128 Len=0
8	0.003501659	192.168.179.134	192.168.179.1	HTTP	3434	HTTP/1.1 200 OK (text/html)
9	0.003842791	192.168.179.1	192.168.179.134	TCP	60	49898 → 80 [ACK] Seq=516 Ack=2921 Win=131328 Len=0
10	0.052351628	192.168.179.1	192.168.179.134	TCP	60	49898 → 80 [ACK] Seq=516 Ack=3381 Win=130816 Len=0
11	5.009821813	192.168.179.134	192.168.179.1	TCP	54	80 → 49898 [FIN, ACK] Seq=3381 Ack=516 Win=64128 Len=0
12	5.010057495	192.168.179.1	192.168.179.134	TCP	60	49898 → 80 [ACK] Seq=516 Ack=3382 Win=130816 Len=0
13	50.010914047	192.168.179.1	192.168.179.134	TCP	60	[TCP Keep-Alive] 49898 → 80 [ACK] Seq=515 Ack=3382 Win=130816 Len=1
14	50.011384589	192.168.179.134	192.168.179.1	TCP	54	[TCP Keep-Alive ACK] 80 → 49898 [ACK] Seq=3382 Ack=516 Win=64128 Len=0

Imagen 4: Captura de trafico de la red

Como se puede apreciar en la captura de red, primero hace un [SYN] [SYN, ACK] [ACK] y establece dicha conexión, pero en vez de enviar y recibir la respuesta en el protocolo 80, Nuestra petición siendo por el puerto 49898, pasa al puerto 80 y nos muestra la pagina. Por lo que gracias a esto podemos saber que hay una subred de servicios, ya que como vimos en el primer escaneo de puertos, ni el puerto 80 ni el 49898 están abiertos.

4.2. Explotación de posibles servicios

Al alojar servicios por defectos de ultima versión, no hay vulnerabilidades conocidas que puedan ser determinantes o afecten a la integridad del servidor, por lo que se realizara un ejercicio de simulación de servicios ofrecidos por dicho servidor con vulnerabilidades simuladas.

Entre algunas de las vulnerabilidades que se van a simular están:

- XSS (Cross Site Scripting)
- SQLI (SQL Injection)
- CI (Command Injection)
- FI (File Inclusion)

Dichas vulnerabilidades podrían ser posibles debido a mala sanitización de inputs por parte del usuario.



5. Contramedidas y buenas practicas

Con el objetivo de evitar posibles explotaciones indeseadas en el servidor expuesto, se enumeran a continuación las buenas prácticas a llevar a cabo para las diferentes posibles vulnerabilidades.

5.1. Vulnerabilidad XSS

Como recomendación, se ofrece utilizar un código similar a este realizado en PHP, en cada input que puede ser susceptible por el usuario:

```
1 <?php
2
3 // Is there any input?
4 if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
5     // Check Anti-CSRF token
6     checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php'
7         );
8
9     // Get input
10    $name = htmlspecialchars( $_GET[ 'name' ] );
11
12    // Feedback for end user
13    echo "<pre>Hello_{$name}</pre>";
14 }
15
16 // Generate Anti-CSRF token
17 generateSessionToken();
18 ?>
```




5.2. Vulnerabilidad SQLI

Como recomendación, se ofrece utilizar un código similar a este realizado en PHP, en cada input que puede ser susceptible por el usuario:

```
1 <?php
2
3 if( isset( $_GET[ 'Submit' ] ) ) {
4     // Check Anti-CSRF token
5     checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php'
6         );
7
8     // Get input
9     $id = $_GET[ 'id' ];
10
11     // Was a number entered?
12     if(is_numeric( $id )) {
13         $id = intval ( $id );
14         switch ( $_DVWA[ 'SQLI_DB' ] ) {
15             case MYSQL:
16                 // Check the database
17                 $data = $db->prepare( 'SELECT first_name, last_name FROM users WHERE
18                     user_id = :id LIMIT 1;' );
19                 $data->bindParam( ':id', $id, PDO::PARAM_INT );
20                 $data->execute();
21                 $row = $data->fetch();
22
23                 // Make sure only 1 result is returned
24                 if( $data->rowCount() == 1 ) {
25                     // Get values
26                     $first = $row[ 'first_name' ];
27                     $last = $row[ 'last_name' ];
28
29                     // Feedback for end user
30                     echo "<pre>ID:{$id}<br/>First name:{$first}<br/>Surname:{$
31                         $last}</pre>";
32                 }
33                 break;
34             case SQLITE:
35                 global $sqlite_db_connection;
36
37                 $stmt = $sqlite_db_connection->prepare( 'SELECT first_name, last_name
38                     FROM users WHERE user_id = :id LIMIT 1;' );
39                 $stmt->bindValue( ':id', $id, SQLITE3_INTEGER );
40                 $result = $stmt->execute();
41                 $result->finalize();
42                 if ( $result !== false ) {
43                     // There is no way to get the number of rows returned
44                     // This checks the number of columns (not rows) just
45                     // as a precaution, but it won't stop someone dumping
46                     // multiple rows and viewing them one at a time.
47
48                     $num_columns = $result->numColumns();
49                     if ( $num_columns == 2 ) {
```



```
46         $row = $result->fetchArray();
47
48         // Get values
49         $first = $row[ 'first_name' ];
50         $last  = $row[ 'last_name' ];
51
52         // Feedback for end user
53         echo "<pre>ID:␣{$id}<br␣/>First␣name:␣{$first}<br␣/>Surname:␣
54             {$last}</pre>";
55     }
56 }
57 break;
58 }
59 }
60 }
61
62 // Generate Anti-CSRF token
63 generateSessionToken();
64
65 ?>
```



5.3. Vulnerabilidad CI

Como recomendación, se ofrece utilizar un código similar a este realizado en PHP, en cada input que puede ser susceptible por el usuario:

```
1 <?php
2
3 if( isset( $_POST[ 'Submit' ] ) ) {
4     // Check Anti-CSRF token
5     checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php'
6         );
7
8     // Get input
9     $target = $_REQUEST[ 'ip' ];
10    $target = stripslashes( $target );
11
12    // Split the IP into 4 octects
13    $octet = explode( ".", $target );
14
15    // Check IF each octet is an integer
16    if( ( is_numeric( $octet[0] ) ) && ( is_numeric( $octet[1] ) ) && ( is_numeric(
17        $octet[2] ) ) && ( is_numeric( $octet[3] ) ) && ( sizeof( $octet ) == 4 ) ) {
18        // If all 4 octets are int's put the IP back together.
19        $target = $octet[0] . '.' . $octet[1] . '.' . $octet[2] . '.' . $octet[3];
20
21        // Determine OS and execute the ping command.
22        if( striistr( php_uname( 's' ), 'Windows_NT' ) ) {
23            // Windows
24            $cmd = shell_exec( 'ping' . $target );
25        }
26        else {
27            // *nix
28            $cmd = shell_exec( 'ping -c 4' . $target );
29        }
30
31        // Feedback for the end user
32        echo "<pre>{$cmd}</pre>";
33    }
34    else {
35        // Ops. Let the user name theres a mistake
36        echo '<pre>ERROR: You have entered an invalid IP.</pre>';
37    }
38 }
39
40 // Generate Anti-CSRF token
41 generateSessionToken();
42
43 ?>
```



5.4. Vulnerabilidad FI

Como recomendación, se ofrece utilizar un código similar a este realizado en PHP, en cada input que puede ser susceptible por el usuario:

```
1 <?php
2
3 // The page we wish to display
4 $file = $_GET[ 'page' ];
5
6 // Only allow include.php or file{1..3}.php
7 $configFileNames = [
8     'include.php',
9     'file1.php',
10    'file2.php',
11    'file3.php',
12 ];
13
14 if( !in_array($file, $configFileNames) ) {
15     // This isn't the page we want!
16     echo "ERROR: File not found!";
17     exit;
18 }
19
20 ?>
```

6. Conclusión

No se han detectado **vulnerabilidades críticas** que puedan suponer un riesgo desde el punto de vista de la seguridad informática. Por ello se han recreado posibles vulnerabilidades que podrían llegar a darse en caso de que los servicios se actualicen y utilicen Inputs de usuario.

Se recomienda encarecidamente aplicar las contramedidas recomendadas para corregir estas vulnerabilidades en caso de que se añadan input, dado que de lo contrario se podría comprometer la seguridad informática del servidor y poner en riesgo la integridad de todos los datos almacenados en este.